

Difficulty-Tiered Fog-of-War Maze Generation via Markov Chain Monte Carlo Sampling

May 2026

1 Introduction and Motivation

Procedural Content Generation (PCG) is the algorithmic creation of game content such as maps, levels, enemies, and items. In grid-based games, the central challenge is not only generating valid maps, but generating maps with controllable and meaningful difficulty. This project focuses on generating difficulty-tiered 2D maze maps for a resource-management escape game.

The proposed game is a grid-based dungeon escape game with fog of war. The player starts from a fixed start cell and must reach an exit door. The door may be initially open, or it may require a key. The map contains walls, one-cell-wide maze corridors, stamina potions, strength potions, and stationary monsters. Each movement consumes stamina. The player starts with zero strength and gains strength by collecting strength items. When the player enters a monster cell, the monster can only be defeated if the player's current strength is at least equal to the monster's strength value. After defeating the monster, that amount is subtracted from the player's strength. The player loses if stamina reaches zero or if they encounter a monster stronger than their current strength.

The map is not fully visible at the beginning. Instead, only cells within a limited visibility radius around the player are revealed. Previously revealed cells remain visible. This makes exploration and resource management central to gameplay.

This project frames map generation as a probabilistic sampling problem. A distribution over valid maps is defined such that maps closer to a target difficulty have higher probability. Markov Chain Monte Carlo (MCMC), specifically the Metropolis–Hastings algorithm, is used to sample maps from this distribution. Difficulty is estimated through Monte Carlo simulation of multiple agent playthroughs under partial map visibility.

2 Problem Definition

We consider an $N \times N$ grid map M , where each cell belongs to one of the following types:

- Wall
- Floor
- Start cell S
- Door cell D
- Key cell K
- Monster cell
- Strength potion cell
- Stamina potion cell

The player state is defined as:

$$s = (p, stamina, strength, hasKey, V, I, R)$$

where:

- p is the player's current position,
- $stamina$ is the player's remaining stamina,
- $strength$ is the player's current strength,
- $hasKey$ indicates whether the key has been collected,
- V is the set of visible or previously revealed cells,
- I is the set of collected items,
- R is the set of defeated monsters.

The initial state is:

$$s_0 = (S, Stamina_0, 0, false, V_0, \emptyset, \emptyset)$$

where V_0 contains the initially visible cells around the start position.

At each step, the player may move to an adjacent non-wall cell. Each movement decreases stamina by one:

$$stamina \leftarrow stamina - 1$$

If the player enters a stamina potion cell, stamina increases by the potion value and the potion disappears. If the player enters a strength potion cell, strength increases by the potion value and the potion disappears.

If the player enters a monster cell with monster strength m , the transition is valid only if:

$$strength \geq m$$

After defeating the monster:

$$strength \leftarrow strength - m$$

If the door is unlocked, reaching the door results in success. If the door is locked, the player must collect the key before entering the door.

The core research question is:

Can Markov Chain Monte Carlo sampling generate fog-of-war maze maps whose difficulty, measured by Monte Carlo agent simulation, can be controlled across multiple difficulty tiers?

3 Objectives and Deliverables

The main objectives of this project are:

1. Implement a grid-based maze map representation.
2. Implement hard constraints for connectivity, one-cell-wide maze structure, and full resource-aware solvability.

3. Implement a resource-aware solver that determines whether a map is theoretically solvable under the full game rules.
4. Implement an MCMC map sampler using the Metropolis–Hastings algorithm.
5. Implement a fog-of-war agent simulator.
6. Estimate map difficulty using Monte Carlo simulation with $K = 500$ independent agent playthroughs.
7. Generate maps across five target difficulty tiers.
8. Analyze convergence, acceptance rate, sampling efficiency, and difficulty calibration.

The expected deliverables are:

- Python source code,
- written project report,
- visual gallery of generated maps across difficulty tiers,
- convergence diagnostic plots,
- difficulty calibration curves,
- success and failure statistics from Monte Carlo simulations.

4 Methodology

4.1 Map Generation via Metropolis–Hastings

Let M denote a candidate map. We define a Boltzmann distribution over valid maps:

$$P(M) \propto \exp\left(-\frac{E(M)}{T}\right)$$

where $E(M)$ is the energy function and T is the temperature. Lower energy corresponds to better maps, meaning maps closer to the desired difficulty target are sampled with higher probability.

At each MCMC step, a candidate map M' is generated by applying a local modification to the current map. Possible proposal moves include:

- converting a wall cell to a floor cell,
- converting a floor cell to a wall cell,
- moving a stamina potion,
- moving a strength potion,
- moving a monster,
- changing a monster’s strength value,
- moving the key,
- toggling whether the door requires a key.

The candidate map is accepted with probability:

$$\alpha = \min\left(1, \exp\left(-\frac{E(M') - E(M)}{T}\right)\right)$$

A simulated annealing schedule may be used to improve convergence:

$$T(t) = T_0 \cdot \exp(-\gamma t)$$

This allows the sampler to explore broadly at the beginning and gradually focus on lower-energy maps.

4.2 Energy Function

The energy function is defined as:

$$E(M) = \lambda_1 E_{path}(M) + \lambda_2 E_{monster}(M) + \lambda_3 E_{resource}(M) + \lambda_4 E_{key}(M) + \lambda_5 E_{exploration}(M) + \lambda_6 E_{difficulty}(M)$$

where each $\lambda_i \geq 0$ is a tunable weight.

4.2.1 Path Term

The path term controls the approximate length of the solution route:

$$E_{path}(M) = \frac{|L_{solution} - L_{target}|}{N^2}$$

where $L_{solution}$ is the length of a valid solution path found by the resource-aware solver, and L_{target} is the target solution length for the selected difficulty tier.

Path length is not treated as a hard constraint. Long paths are allowed because they naturally increase stamina consumption and are reflected in both the resource-aware solvability check and the Monte Carlo difficulty estimate.

4.2.2 Monster Term

The monster term controls monster pressure:

$$E_{monster}(M) = \frac{|G_{monster} - G_{target}|}{N^2}$$

where $G_{monster}$ is the total strength cost imposed by monsters on important routes, and G_{target} is the desired monster pressure for the selected difficulty tier.

4.2.3 Resource Term

The resource term controls the amount and placement of stamina and strength resources:

$$E_{resource}(M) = |R_{stamina} - R_{stamina}^*| + |R_{strength} - R_{strength}^*|$$

where $R_{stamina}$ and $R_{strength}$ are the available stamina and strength resources, and $R_{stamina}^*$ and $R_{strength}^*$ are tier-specific targets.

4.2.4 Key-Door Term

If the door is locked, the key-door term measures the additional route complexity introduced by the key:

$$E_{key}(M) = \frac{dist(S, K) + dist(K, D)}{N^2}$$

If the door is unlocked, this term is set to zero.

4.2.5 Exploration Term

The exploration term captures how much the player is encouraged to explore under fog of war:

$$E_{\text{exploration}}(M) = \frac{\text{deadEndCount}(M)}{N^2}$$

Higher difficulty tiers may target more dead ends and misleading branches, while easier tiers may prefer more direct maze layouts.

4.2.6 Difficulty Term

The difficulty term steers the sampler toward the desired Monte Carlo difficulty estimate:

$$E_{\text{difficulty}}(M) = (\hat{D}(M) - d_{\text{target}})^2$$

where $\hat{D}(M)$ is the estimated failure rate from Monte Carlo agent simulation, and:

$$d_{\text{target}} \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$$

corresponds to five increasing difficulty tiers.

4.3 Hard Constraints

Hard constraints are enforced before evaluating the energy function. If a candidate map violates any hard constraint, it is immediately rejected.

4.3.1 HC1 — Full Connectivity

All non-wall cells must be reachable from the start cell through walkable cells. This ensures that the map contains no unreachable corridors, items, monsters, keys, or doors.

4.3.2 HC2 — One-Cell-Wide Maze Structure

The map must preserve a maze-like corridor structure. Candidate maps containing 2×2 open-cell blocks are rejected. This prevents wide rooms or wide corridors and keeps the map visually and structurally similar to a traditional maze.

4.3.3 HC3 — Resource-Aware Solvability

There must exist at least one valid solution path from the start cell to the door under the full game rules. This constraint jointly considers:

- stamina consumption per movement,
- stamina potion collection,
- strength potion collection,
- monster strength costs,
- key collection,
- locked-door requirements.

This replaces separate stamina, strength, key, and monster feasibility checks. These factors must be evaluated together because separate checks may incorrectly accept maps where one path is feasible in terms of stamina and another path is feasible in terms of strength, but no single path satisfies all requirements simultaneously.

4.3.4 HC4 — Tier-Dependent Minimum Resource Margin

A valid solution path must satisfy a minimum resource safety margin. The margin is high for easier tiers and low for more difficult tiers.

For example, the solution path may be required to satisfy:

$$stamina_{final} \geq margin_{stamina}$$

and:

$$strength_{final} \geq margin_{strength}$$

The margins can be defined as:

Difficulty Tier	Stamina Margin	Strength Margin
Very Easy	High	High
Easy	Medium-High	Medium
Medium	Medium	Low
Hard	Low	Very Low
Very Hard	Zero or Near-Zero	Zero or Near-Zero

This allows easy maps to have a larger error buffer, while hard maps may be solvable only with tight resource management.

4.4 Resource-Aware Solver

A resource-aware solver is used to determine whether a candidate map is theoretically solvable. Unlike a standard BFS, which only tracks position, this solver searches an expanded state space:

$$s = (p, stamina, strength, hasKey, I, R)$$

where I is the set of collected items and R is the set of defeated monsters.

The solver starts from:

$$s_0 = (S, Stamina_0, 0, false, \emptyset, \emptyset)$$

and explores valid transitions according to the game rules. A transition to a neighboring cell is valid only if:

- the target cell is not a wall,
- the player has positive stamina before movement,
- the player can pay the monster strength cost if the target contains an undefeated monster,
- the player has the key if the target is a locked door.

The map is accepted by HC3 if the solver finds at least one successful state in which the player reaches the door.

The solver is not used to measure difficulty. It is used only as a hard validity filter. Difficulty is still estimated statistically through Monte Carlo simulation of agents under partial observability.

To reduce computation time, dominance pruning can be used. If two states share the same position and key status, and one state has greater or equal stamina and strength than the other, the weaker state can be discarded.

4.5 Monte Carlo Agent Simulation

For each valid candidate map, difficulty is estimated by simulating $K = 500$ independent agent playthroughs.

Each simulated agent has the same game rules as the player:

- movement consumes stamina,
- stamina potions increase stamina,
- strength potions increase strength,
- monsters consume strength,
- locked doors require a key,
- the agent loses if stamina reaches zero or if it encounters a monster stronger than its current strength.

The agent operates under fog of war. At the beginning, only cells within a fixed visibility radius are known. As the agent moves, new cells are revealed, and previously revealed cells remain visible.

The agent follows an ϵ -greedy policy:

$$\pi(a|s) = \begin{cases} \text{best known action,} & \text{with probability } 1 - \epsilon \\ \text{random valid action,} & \text{with probability } \epsilon \end{cases}$$

The best known action depends on the currently visible and remembered map:

- if the visible door is unlocked, move toward the door;
- if the visible door is locked and the key has not been collected, search for the key;
- if the key is visible, move toward the key;
- if a useful stamina or strength item is visible, move toward it if beneficial;
- otherwise, move toward an unexplored frontier cell.

A simulation run is successful if the agent reaches the door under the correct condition. A run is considered failed if:

- stamina reaches zero,
- the agent encounters a monster stronger than its current strength,
- the agent cannot find a valid move,
- a maximum simulation step limit is exceeded.

The maximum step limit is used only to prevent infinite wandering during simulation. It is not a hard constraint on map generation. For example:

$$\text{maxSteps} = cN^2$$

where c may be set to 5 or 8.

The difficulty estimate is:

$$\hat{D}(M) = \frac{\text{failed runs}}{K}$$

The uncertainty of this estimate can be approximated by:

$$\sigma_{\hat{D}} = \sqrt{\frac{\hat{D}(M)(1 - \hat{D}(M))}{K}}$$

This nested structure is the core of the project: MCMC is used for map generation, while Monte Carlo simulation is used for difficulty estimation.

4.6 Difficulty Tiers

The system generates maps for five difficulty tiers:

Tier	Target Difficulty
Very Easy	$d_{target} = 0.1$
Easy	$d_{target} = 0.3$
Medium	$d_{target} = 0.5$
Hard	$d_{target} = 0.7$
Very Hard	$d_{target} = 0.9$

Each tier adjusts target properties such as:

- solution path length,
- number of dead ends,
- monster count,
- total monster strength,
- stamina potion availability,
- strength potion availability,
- key distance,
- resource safety margin.

Easy maps should have shorter paths, fewer monsters, more accessible resources, and larger stamina and strength margins. Hard maps should have longer paths, more misleading branches, stronger mandatory monsters, tighter stamina margins, and more difficult key-door placement.

4.7 Computational Considerations

The resource-aware solver is more expensive than ordinary BFS because it searches over position, stamina, strength, item collection, monster defeat status, and key possession. However, it is still practical for moderate grid sizes such as 15×15 , 21×21 , or 25×25 , especially when item and monster counts are bounded.

The pipeline first applies cheap validity checks, such as full connectivity and the 2×2 open-block constraint. The resource-aware solver is only run on maps that pass these preliminary checks.

The most computationally expensive part of the system is expected to be the Monte Carlo difficulty estimation, since it may require hundreds of agent simulations for each valid candidate map. If necessary, a two-stage estimation strategy can be used:

- use a smaller number of simulations during MCMC sampling,
- use $K = 500$ simulations for final evaluation and reporting.

4.8 Importance Sampling for Extreme Difficulty Tiers

Very easy and very hard maps may be rare under the standard proposal distribution. Importance Sampling can be used to improve sampling efficiency for extreme target tiers.

For very easy maps, the proposal distribution may be biased toward:

- shorter solution paths,
- more stamina potions,

- more strength potions,
- fewer monsters,
- unlocked doors,
- fewer dead ends.

For very hard maps, the proposal distribution may be biased toward:

- longer paths,
- locked doors,
- more dead ends,
- stronger monsters,
- tighter resource margins,
- more distant key placement.

Sampling efficiency can be evaluated using Effective Sample Size (ESS).

5 Evaluation Plan

The project will be evaluated using both qualitative and quantitative analysis.

5.1 Generated Map Gallery

A visual gallery will show generated maps for each difficulty tier. The maps should demonstrate increasing complexity, longer routes, more dangerous monster placement, and tighter resource availability as difficulty increases.

5.2 Difficulty Calibration

For each target difficulty tier, multiple maps will be generated and evaluated using Monte Carlo agent simulations. The measured difficulty $\hat{D}(M)$ will be compared with the target difficulty d_{target} .

A successful result would show that generated maps cluster near their intended target difficulty levels.

5.3 MCMC Diagnostics

The MCMC sampling process will be evaluated using:

- energy over time,
- acceptance rate,
- difficulty estimate over time,
- convergence behavior,
- number of rejected candidates due to hard constraints.

5.4 Agent Failure Analysis

Agent failures will be categorized by cause:

- stamina depletion,
- insufficient strength against a monster,
- failure to find the key,

- failure to reach the door within the step limit,
- inefficient exploration under fog of war.

This analysis will help explain why maps are difficult and whether the generated difficulty is meaningful.

6 Expected Outcome

The expected outcome is a complete procedural generation pipeline capable of producing playable fog-of-war maze maps across multiple difficulty tiers. Every generated map will satisfy strict validity constraints, including full connectivity, one-cell-wide maze structure, and resource-aware solvability. Among these valid maps, difficulty will be controlled using Monte Carlo simulation of agent playthroughs.

The final system should demonstrate that MCMC sampling combined with Monte Carlo difficulty estimation can generate maps that are not only valid, but also progressively more challenging according to a measurable difficulty metric.

7 Conclusion

This project combines procedural content generation, Markov Chain Monte Carlo sampling, resource-aware path solving, and Monte Carlo simulation-based difficulty estimation. The resource-aware solver guarantees that generated maps are theoretically solvable under the complete game rules. The Monte Carlo agent simulation then estimates how difficult each solvable map is under partial observability and imperfect exploration.

This separation is important: the solver determines validity, while Monte Carlo simulation determines difficulty. Therefore, the use of a deterministic solver does not replace the Monte Carlo component. Instead, it strengthens the framework by ensuring that high-difficulty maps are genuinely challenging rather than impossible.